

CHAPTER 04

04. Express 입문 — 첫 JSON API 서버

Node.js · Express · NestJS 강의 · 2026

브라우저 주소창에 `localhost:3000` 을 치면 무언가 응답이 돌아온다. 그 "무언가"를 만들어 돌려주는 프로그램이 **웹 서버**다. 이 장에서는 Node.js의 대표 웹 프레임워크인 **Express**로, 데이터를 JSON으로 주고받는 가장 작은 서버부터 게시물 만들·고치·지우는 API까지 직접 만들어 본다.

화면(HTML)은 다루지 않는다. 요즘 웹은 서버가 **데이터(JSON)** 만 내려주고, 화면은 React 같은 프론트엔드가 그리는 구조가 표준이다. 그래서 이 과정의 서버는 처음부터 끝까지 화면 없는 **JSON API**다.

이 자료는 **개념 연습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- Express 서버의 뼈대: `require` → `app` → 라우트 → `listen`
- 요청에서 값을 꺼내는 세 자리: **경로 파라미터**(`:id`), **쿼리스트링**(`?author=`), **본문**(`req.body`)
- HTTP 메서드(GET·POST·PUT·DELETE)와 상태코드(200·201·400·404)를 CRUD에 맞춰 쓰는 법
- 미들웨어가 무엇이고 `next()` 가 왜 중요한지

내용은 네 주제로 나뉘고, 뒤로 갈수록 개념이 하나씩 쌓인다.

주제	핵심
가장 작은 서버	서버를 띄우는 최소 코드
읽기	목록 · 단건 · 검색
쓰기	생성 · 수정 · 삭제
미들웨어	요청 파이프라인과 <code>next()</code>

핵심 개념 — 서버는 "요청을 받아 응답을 돌려주는" 함수다

웹 서버가 하는 일은 한 문장으로 요약된다. **요청(request)**이 들어오면, 그에 맞는 **응답(response)**을 돌려준다. 클라이언트(브라우저, 앱, `curl`)가 "이 주소로 이런 메서드의 요청을 보낸다"고 하면, 서버는 "그럼 이 데이터를 이 상태코드로 돌려준다"고 답한다.

요청에는 두 가지 핵심 정보가 있다.

- **메서드(method)** — 무엇을 하려는가. `GET` (읽기), `POST` (생성), `PUT` (수정), `DELETE` (삭제)
- **경로(path)** — 무엇에 대해. `/posts` (게시물 목록), `/posts/1` (1번 게시물)

이 둘의 조합 하나하나를 **라우트(route)** 라고 한다. GET /posts 와 POST /posts 는 경로가 같아도 메서드가 다르니 다른 라우트다. Express는 결국 "어떤 메서드 + 어떤 경로 → 어떤 함수를 실행할지"를 등록하고 연결한다. 그 연결된 함수를 **핸들러(handler)** 라고 부른다.

왜 Express인가 — Node.js만으로도 서버를 만들 수 있지만(2장의 http 모듈), 경로마다 분기하고 본문을 파싱하는 코드를 매번 직접 짜야 한다. Express는 그 반복을 `app.get()`, `express.json()` 같은 짧은 형태로 정리한다. 가장 널리 쓰이는 Node 웹 프레임워크이고, 뒤에 배울 NestJS도 내부적으로 Express 위에서 동작한다.

준비

예제 폴더에서 `npm install` 로 의존성을 설치한다(이 장에선 `express` 하나뿐). 그다음 서버 파일을 실행하면 된다.

서버가 뜨면 터미널이 그 자리에서 멈춘 것처럼 보인다. 멈춘 게 아니라 **요청을 기다리는 중이다**. 다른 터미널을 열어 `curl` 로 요청을 보내거나 브라우저로 접속하면 응답이 온다. 한 번에 하나씩 실행하고, 종료는 `Ctrl+C` 다.

가장 작은 서버 — 라우트와 응답

```
const express = require("express");
const app = express();

// GET / 요청이 오면 JSON으로 응답한다.
app.get("/", (req, res) => {
  res.json({ message: "Hello, Express!" });
});

// 3000 포트에서 서버를 띄운다.
app.listen(3000, () => {
  console.log("http://localhost:3000 에서 실행 중");
});
```

단 네 줄짜리 동작이지만, 모든 Express 서버가 이 뼈대를 공유한다. 한 줄씩 뜯어보자.

- `const express = require("express")` — 설치한 `express` 모듈을 불러온다. 이때 `express` 는 함수다.
- `const app = express()` — 그 함수를 호출하면 **애플리케이션 객체** `app` 이 만들어진다. 앞으로 모든 라우트와 설정은 이 `app` 에 등록한다.
- `app.get("/", (req, res) => {...})` — "GET 메서드로 / 경로에 요청이 오면 이 함수를 실행하라"는 등록이다. 핸들러는 두 인자를 받는다.
 - `req` (request) — 들어온 요청. 주소, 헤더, 쿼리, 본문 등이 들어 있다.
 - `res` (response) — 돌려줄 응답. 여기에 무엇을 담아 보낼지 우리가 정한다.

- `res.json({..})` — 객체를 **JSON 문자열로 바꿔** 응답 본문에 싣고, `Content-Type: application/json` 헤더까지 자동으로 붙여 보낸다. API 서버에서 가장 많이 쓰는 응답 메서드다.
- `app.listen(3000, ...)` — 3000번 포트에서 요청을 받기 시작한다. 이 줄이 있어야 비로소 서버가 "켜진다". 콜백은 서버가 준비됐을 때 한 번 실행된다.

요청 한 번의 흐름을 그림으로 옮기면 이렇다.

브라우저 — GET / —▶ app —▶ ("/", GET) 핸들러 실행 —▶ `res.json(.)` —▶ 브라우저

짚고 가기 — `app.get` 의 `get` 은 HTTP 메서드 이름이다. `app.post`, `app.put`, `app.delete` 도 똑같은 모양으로 쓴다. 뒤에서 바로 등장한다.

읽기 — 목록·단건·검색

이제 돌려줄 데이터가 생긴다. 데이터베이스는 6장에서 붙이고, 지금은 배열 하나를 메모리에 두고 쓴다.

```
const express = require("express");
const app = express();

// 인메모리 데이터 — 서버를 끄면 사라진다 (학습용).
const posts = [
  { id: 1, title: "첫 번째 글", content: "안녕하세요", author: "김철수" },
  { id: 2, title: "두 번째 글", content: "반갑습니다", author: "이영희" },
  { id: 3, title: "세 번째 글", content: "또 만나요", author: "김철수" },
];
```

`posts` 는 그냥 자바스크립트 배열이다. 서버를 끄면 초기화되므로 "진짜 저장"은 아니지만, 라우팅을 익히는 데는 충분하다.

목록 조회와 검색 — 쿼리스트링

```
app.get("/posts", (req, res) => {
  const { author } = req.query; // 쿼리스트링: /posts?author=김철수
  if (author) {
    return res.json(posts.filter((p) => p.author === author));
  }
  res.json(posts);
});
```

`/posts` 로 들어온 `GET` 요청을 처리한다. 핵심은 `req.query` 다. 주소 뒤에 `?author=김철수` 처럼 붙는 부분을 쿼리스트링이라 하고, Express는 이를 객체로 파싱해 `req.query` 에 담는다. 여기서는 `{ author: "김철수" }` 가 된다.

- `author` 가 있으면 → 작성자가 일치하는 글만 `filter` 로 추려 응답한다.
- 없으면(`/posts` 만 요청) → 전체 목록을 응답한다.

`return res.json(.)` 에서 `return` 을 붙인 이유가 중요하다. 응답을 보낸 뒤 아래 `res.json(posts)` 까지 실행되면 한 요청에 응답을 두 번 보내려다 에러가 난다. `return` 으로 핸들러를 거기서 끝낸다.

짚고 가기 — 쿼리스트링은 "목록을 거르거나 정렬하는 선택 조건"에 쓴다. 검색(`?author=`), 페이지네이션(`?page=2`), 정렬(`?sort=newest`)이 모두 같은 자리다.

단건 조회 — 경로 파라미터와 404

```
app.get("/posts/:id", (req, res) => {
  const id = Number(req.params.id);
  const post = posts.find((p) => p.id === id);
  if (!post) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  res.json(post);
});
```

경로의 `:id` 부분이 **경로 파라미터**다. `/posts/2` 로 요청하면 `req.params` 가 `{ id: "2" }` 가 된다.

- `Number(req.params.id)` — 경로 파라미터는 **항상 문자열**이다. 배열의 `id` 는 숫자이므로, `"2" === 2` 가 `false` 가 되지 않도록 숫자로 변환한 뒤 비교한다. 자주 놓치는 부분이다.
- `posts.find(.)` — 해당 `id` 의 글을 찾는다. 없으면 `undefined` .
- `res.status(404)` — 못 찾았으면 상태코드 **404 Not Found**를 명시한다. `.json(.)` 과 체이닝해서 본문도 함께 보낸다. 상태코드를 안 정하면 기본값 200(성공)이 나가서, 클라이언트가 "성공했는데 내용이 없다"고 오해한다.

요청 자리	어디서 꺼내나	예시	용도
경로 파라미터	<code>req.params</code>	<code>/posts/2</code> → <code>{ id: "2" }</code>	무엇을 가리킬 때 (특정 자원)
쿼리스트링	<code>req.query</code>	<code>/posts?author=김철수</code>	어떻게 거를 때 (검색·정렬·페이지)

쓰기 — 생성·수정·삭제

읽기만으로는 게시판이 안 된다. 이제 **요청 본문(body)** 으로 데이터를 받아 바꾼다.

```
const express = require("express");
const app = express();

// JSON 본문 파싱 미들웨어 - POST/PUT의 req.body를 쓰려면 필수.
app.use(express.json());

let posts = [
  { id: 1, title: "첫 번째 글", content: "안녕하세요", author: "김철수" },
];
let nextId = 2; // 새 글에 줄 id
```

새로 등장한 줄은 `app.use(express.json())` 다. 클라이언트가 `POST` 로 보낸 JSON 본문은 그냥 두면 글자 덩어리일 뿐이다. `express.json()` 은 그 본문을 읽어 자바스크립트 객체로 바꿔 `req.body` 에 넣는다. 이 줄이 없으면 `req.body` 는 `undefined` 이고, 아래 코드가 전부 깨진다. (이게 바로 잠시 뒤에 배울 *미들웨어*다 — 곧 다시 만난다.)

생성 — POST와 201

```
app.post("/posts", (req, res) => {
  const { title, content, author } = req.body;
  if (!title || !content) {
    return res.status(400).json({ message: "title과 content는 필수입니다." });
  }
  const post = { id: nextId++, title, content, author: author || "익명" };
  posts.push(post);
  res.status(201).json(post);
});
```

- `req.body` 에서 필요한 필드를 구조분해로 꺼낸다.
- `title` 이나 `content` 가 비었으면 **400 Bad Request** — "요청이 잘못됐다"는 신호다. 서버 잘못(500)과 구분된다.
- `author || "익명"` — 작성자를 안 보냈으면 기본값을 채운다.
- 성공 응답은 **201 Created** — "새 자원을 만들었다"는 뜻의 상태코드다. 단순 성공(200)보다 정확하다. 만든 객체를 그대로 돌려줘서, 클라이언트가 부여된 `id` 를 바로 알 수 있게 한다.

수정 — PUT과 부분 변경

```
app.put("/posts/:id", (req, res) => {
  const post = posts.find((p) => p.id === Number(req.params.id));
  if (!post) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  const { title, content, author } = req.body;
  if (title !== undefined) post.title = title;
  if (content !== undefined) post.content = content;
  if (author !== undefined) post.author = author;
  res.json(post);
});
```

`=== undefined` 로 검사하는 게 요령이다. 보낸 필드만 바꾸고, 안 보낸 필드는 그대로 둔다. 만약 `if (title)` 로만 검사하면 빈 문자열 `" "` 로 제목을 지우려는 의도가 무시된다. "값을 보냈는가"와 "값이 참인가"는 다르다.

삭제 — DELETE와 splice

```
app.delete("/posts/:id", (req, res) => {
  const id = Number(req.params.id);
  const index = posts.findIndex((p) => p.id === id);
  if (index === -1) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  // splice는 삭제한 요소들을 배열로 반환 → [removed]로 그 첫 요소를 꺼낸다.
  const [removed] = posts.splice(index, 1);
  res.json({ message: "삭제됨", post: removed });
});
```

- `findIndex` 로 위치를 찾고, 없으면 `-1` 이므로 404.
- `posts.splice(index, 1)` — `index` 자리에서 1개를 들어낸다. `splice` 는 들어낸 요소들의 배열을 반환하므로, `const [removed] =` 로 그 첫 요소를 꺼내 응답에 담는다.

짚고 가기 — 네 메서드가 CRUD에 그대로 대응한다. **Create**= POST , **Read**= GET , **Update**= PUT , **Delete**= DELETE . 이 대응이 REST API 설계의 기본 규칙이다.

미들웨어 — `next()` 로 이어지는 파이프라인

```
const express = require("express");
const app = express();

// 직접 만든 로깅 미들웨어 - 모든 요청이 이 함수를 먼저 거친다.
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // next()를 불러야 다음 단계(라우트)로 넘어간다. 안 부르면 응답이 멈춘다.
});

const posts = [{ id: 1, title: "첫 번째 글", author: "김철수" }];

app.get("/posts", (req, res) => {
  res.json(posts);
});
```

미들웨어(middleware)는 요청과 최종 핸들러 사이에 끼어 동작하는 함수다. 라우트 핸들러가 `(req, res)`를 받았다면, 미들웨어는 인자가 하나 더 있다 — `next`.

- `app.use(fn)` — 모든 요청이 라우트에 닿기 전에 `fn`을 먼저 거치게 한다.
- 이 예제의 미들웨어는 요청이 올 때마다 메서드와 주소를 로그로 남긴다. `GET /posts`로 요청하면 터미널에 `GET /posts`가 찍힌다.
- `next()` — "내 일은 끝났으니 다음 단계로 넘겨라"는 신호다. 이걸 부르지 않으면 요청은 이 미들웨어에 갇혀 라우트까지 가지 못하고, 클라이언트는 응답을 영영 못 받고 멈춘다.

미들웨어는 요청이 지나가는 **파이프라인의 한 칸**이다. 등록된 순서대로 위에서 아래로 실행된다.

요청 → [로깅 미들웨어] —`next()`→ [GET /posts 핸들러] → 응답

이제 앞서 본 `app.use(express.json())`이 다시 보인다. 그것도 미들웨어였다. 본문을 파싱해 `req.body`에 채운 뒤 `next()`로 넘기는 미들웨어다. 우리가 직접 만든 로깅 미들웨어와 정확히 같은 구조다. Express의 많은 기능이 이렇게 미들웨어로 끼워 넣는 형태로 동작한다.

HTTP 메서드 · 상태코드 한눈에

메서드	의미	성공 상태코드
GET	읽기	200 OK
POST	생성	201 Created
PUT	수정	200 OK
DELETE	삭제	200 OK

상태코드	뜻	언제
200	성공	조회·수정·삭제 성공
201	생성됨	POST 로 새 글 생성
400	잘못된 요청	필수 필드 누락 (클라이언트 잘못)
404	찾을 수 없음	없는 id 조회·수정·삭제

흔한 실수

- `express.json()` 을 빼먹는다. → POST / PUT 에서 `req.body` 가 `undefined` . "왜 본문이 안 들어오지?"의 90%는 이것이다.
- 미들웨어에서 `next()` 를 안 부른다. → 요청이 멈춰 응답이 안 온다(타임아웃).
- 경로 파라미터를 문자열인 채 비교한다. → `req.params.id` 는 "2" . `Number()` 로 바꾸지 않으면 `find` 가 못 찾는다.
- 상태코드를 안 정한다. → 못 찾았는데 200이 나가 클라이언트가 오해한다. 실패엔 4xx를 명시한다.
- 응답 뒤 `return` 을 빠뜨린다. → 한 요청에 `res` 를 두 번 보내 `Cannot set headers after they are sent` 에러가 난다.

직접 해보기

1. 목록 검색을 제목 부분 일치로 바꿔 보자. `?q=만나` 로 "또 만나요"가 걸리게 (`p.title.includes(q)`).
2. 글을 하나 POST 로 만든 뒤 GET /posts 로 목록에 추가됐는지 확인하자.
3. 로깅 미들웨어가 응답까지 걸린 시간도 찍게 만들어 보자. 힌트: 미들웨어 시작에서 `Date.now()` 를 기록하고, `res.on("finish", ...)` 에서 차이를 출력.

정리

- Express 서버는 `require` → `app` → 라우트 → `listen` 의 뼈대를 공유한다.
- 요청의 값은 세 자리에서 꺼낸다: `req.params` (경로의 자원 식별), `req.query` (검색·정렬 조건), `req.body` (보낼 데이터).
- CRUD는 `GET` · `POST` · `PUT` · `DELETE` 에 대응하고, 결과는 알맞은 상태코드(200·201·400·404)로 알린다.
- 미들웨어는 요청 파이프라인의 한 칸이며, `next()` 로 다음 칸에 넘긴다. `express.json()` 도 그중 하나다.

다음 장(**05. Express 중급**)에서는 여기서 흩어져 있던 에러 처리를 한곳으로 모으고, 라우트를 자원별로 나누며, 입력 검증을 라이브러리로 단단하게 만든다.